SCHOOL OF
INTERNATIONAL & INTERDISCIPLINARY
ENGINEERING PROGRAMS
King Mongkut's Institute of Technology Ladkrabang

Depth-First Search

Step-by-Step Graph Traversal in Python

Generated by Gemini, Google (21/02/2026)

Edited by

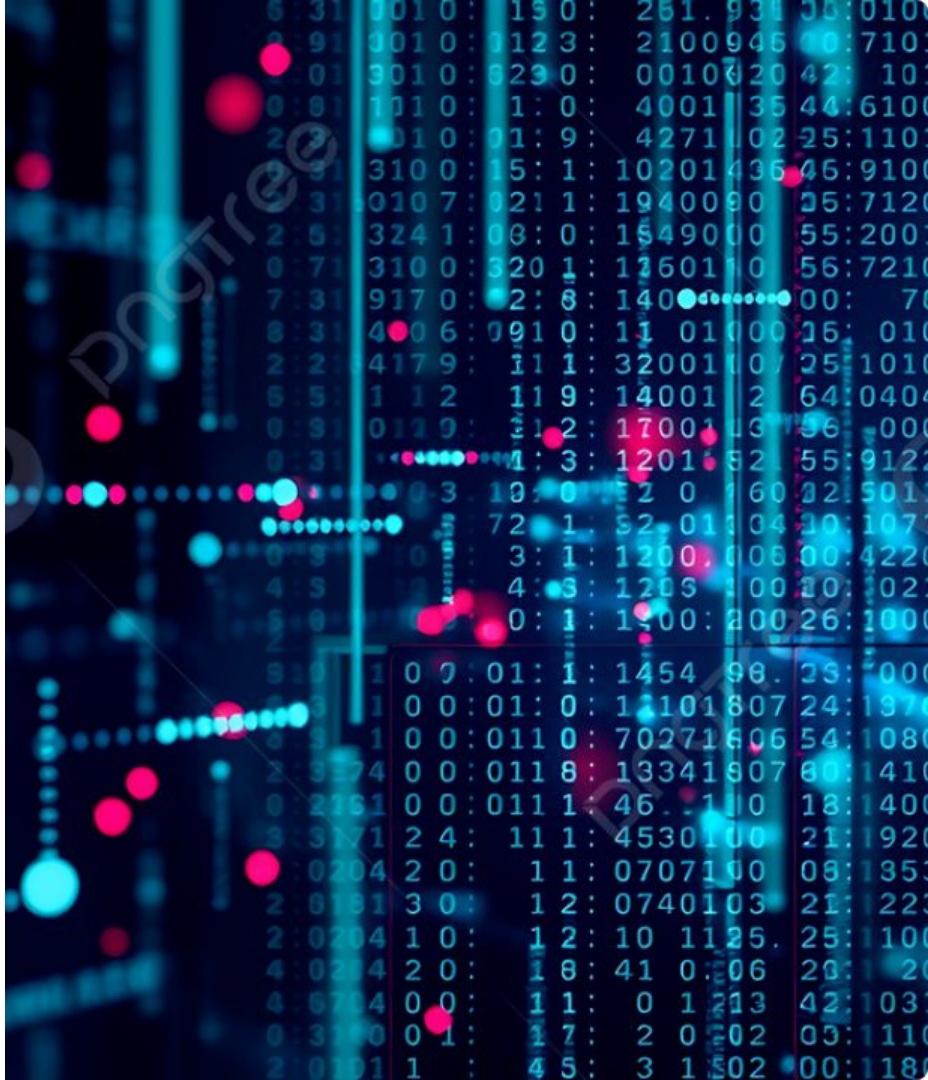
Dr. Sirasit Lochanachit

Tracking DFS State

The core of a Python DFS algorithm relies on two essential data structures to manage the traversal logically:

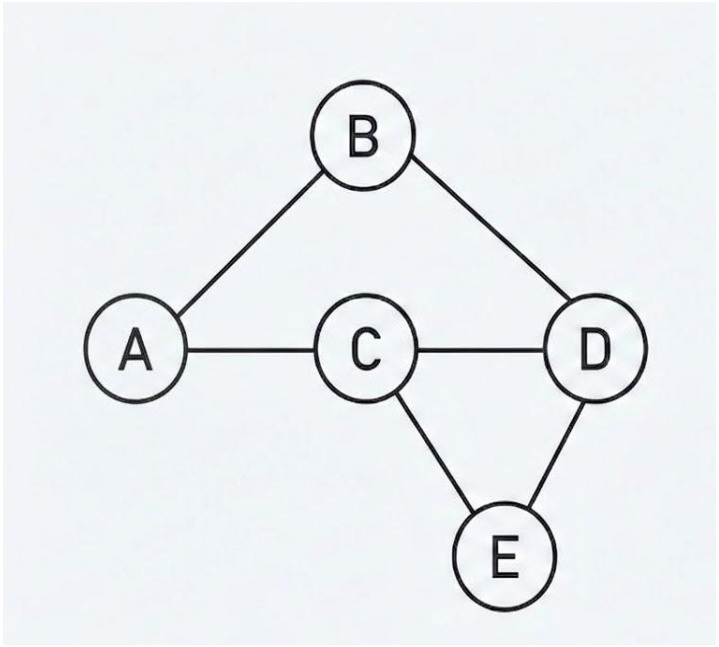
- **visited (Set)**
  Keeps track of accessed nodes to prevent infinite loops in cyclic graphs. Sets offer an average O(1) lookup time.
- **result (List)**
  Records the precise order in which edges are traversed. Lists preserve insertion order perfectly.

We'll trace a simple graph to observe how backtracking pushes algorithm progression.



Depth-First Traversal with Recursion

Our Example Graph



Initial State

Action:

We prepare to start the algorithm. Both data structures are empty. We will invoke dfs('A'). Note: Neighbors are sorted alphabetically.

visited (Set):

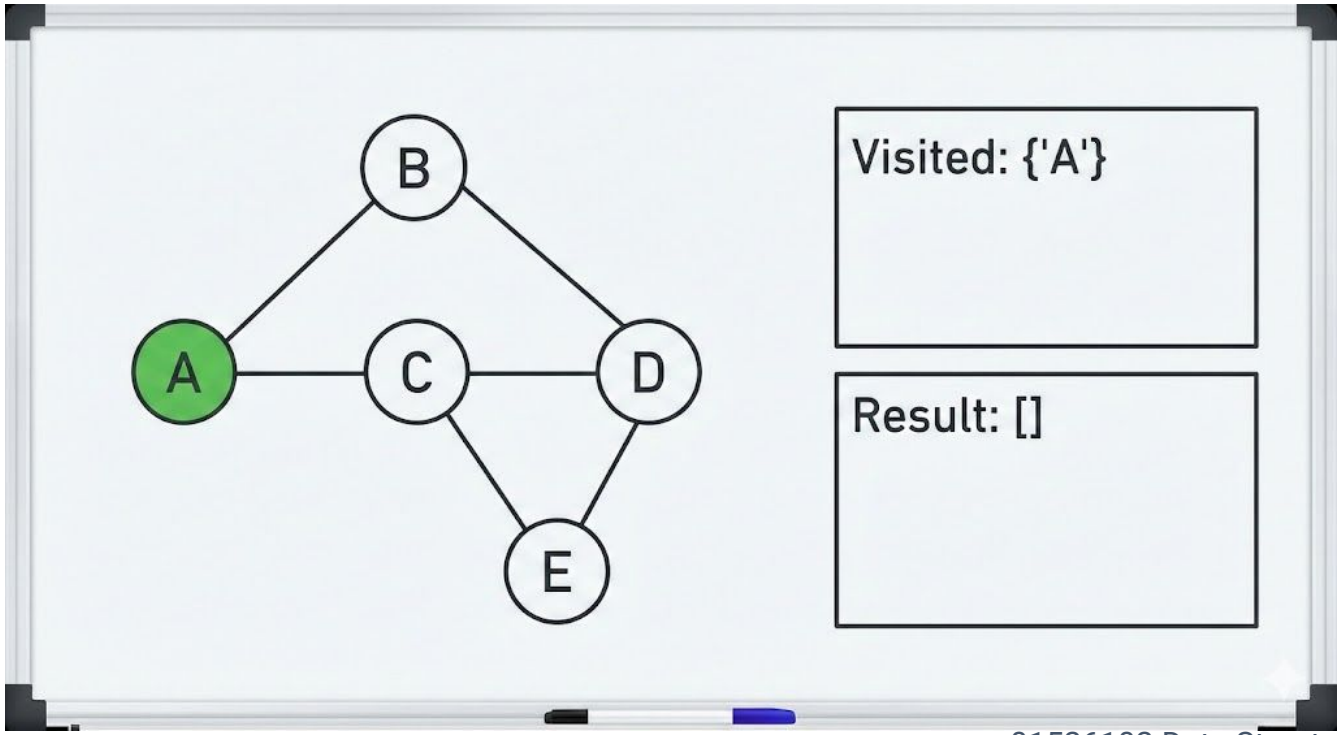
```
set()
```

result (List):

```
[]
```

Step 1: Start at ‘A’

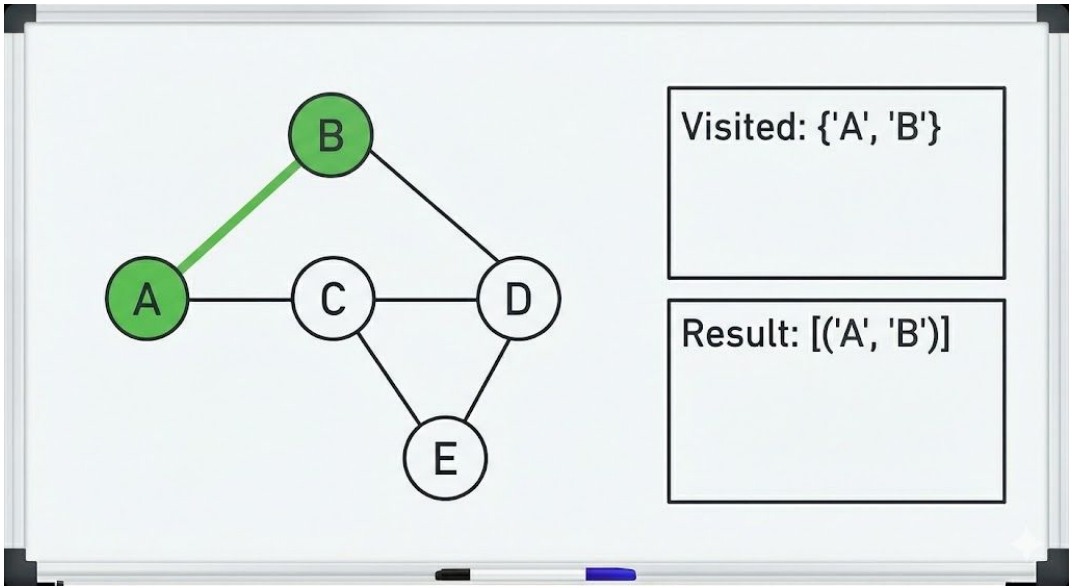
The algorithm begins by calling dfs('A'). The visited set is initialized to {'A'}, and the result list is []. Node 'A' is marked as visited.



Step 2: Visit ‘B’

From 'A', the algorithm looks at its neighbors [B, C]:

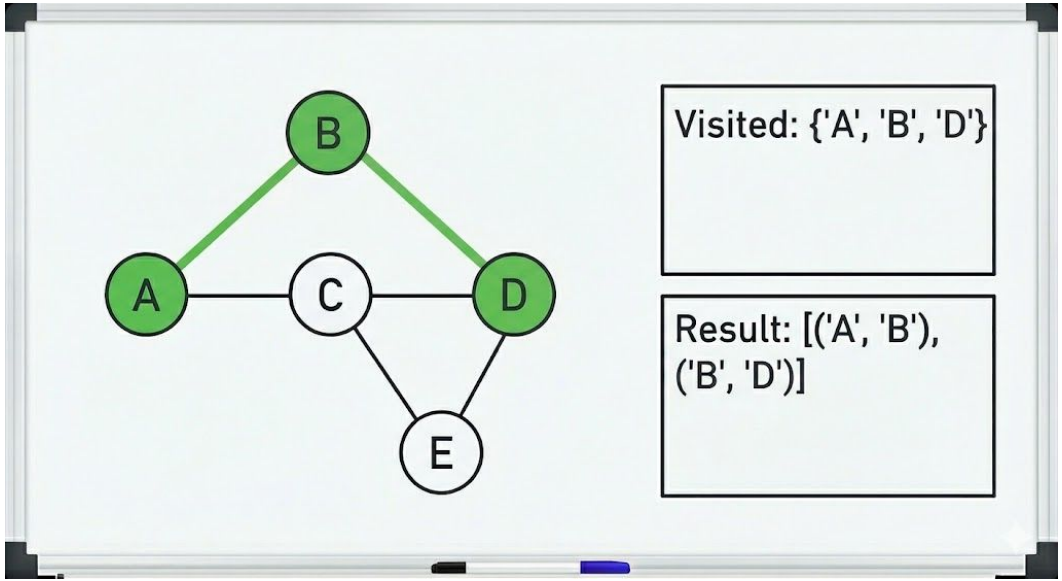
- The code sorts them alphabetically, so it first considers 'B'.
- Since 'B' has not been visited, the algorithm recursively calls dfs_recursive('B', 'A').
- 'B' is marked as visited, and the pair ('A', 'B') is added to the result list.



Step 3: Visit ‘D’

Now at 'B', the algorithm explores its neighbors [A, D]:

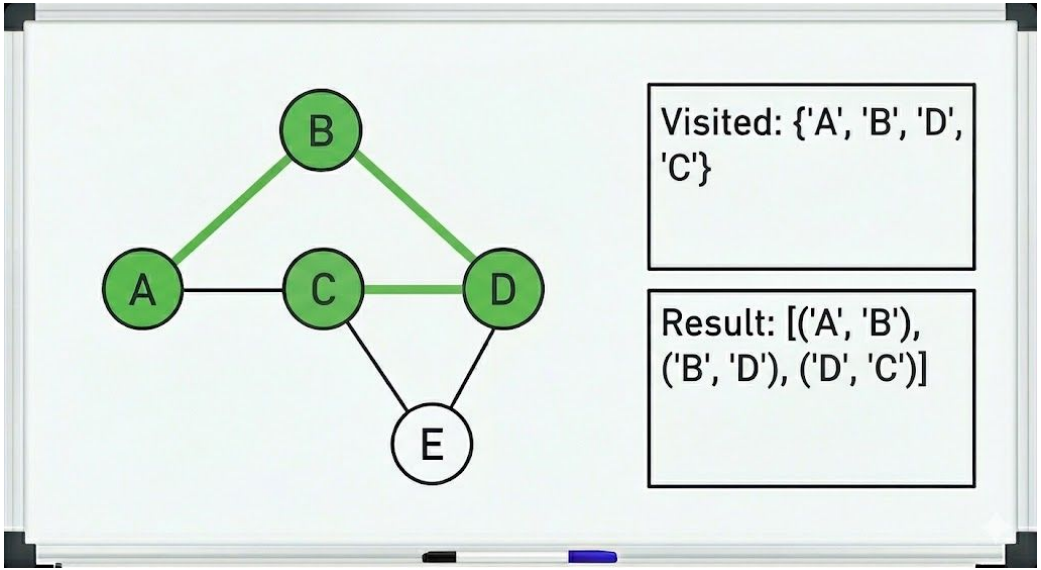
- 'A' is skipped because it is already visited.
- The algorithm then considers 'D'.
- Since 'D' is not visited, it recursively calls dfs_recursive('D', 'B').
- 'D' is marked as visited, and ('B', 'D') is added to the result list.



Step 4: Visit ‘C’

From 'D', the algorithm explores its neighbors [B, C, E]:

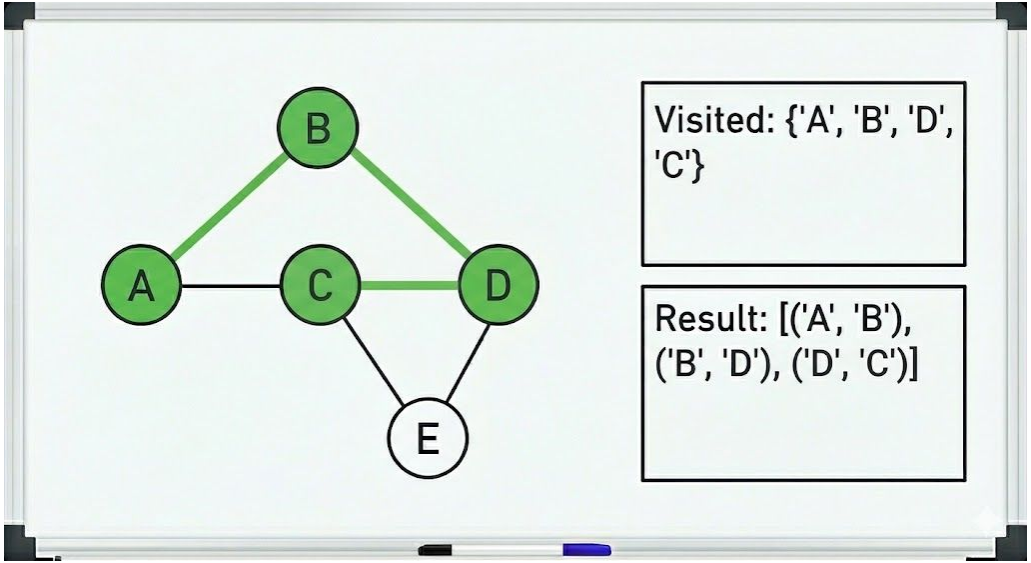
- 'B' is already visited. The next neighbor in sorted order is 'C'.
- Since 'C' is not visited, the algorithm recursively calls dfs_recursive('C', 'D').
- 'C' is marked as visited, and ('D', 'C') is added to the result list.



Step 5: Visit ‘E’ and Detect Back-Edges

Now at ‘C’, the algorithm explores it neighbors [A, D, E]:

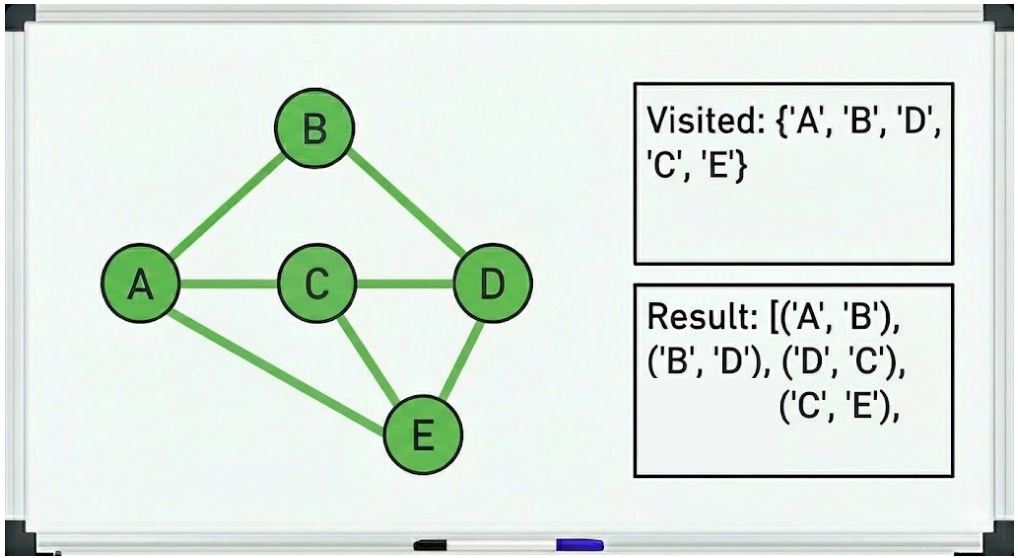
- ‘A’ is visited. The code checks in Result if ‘A’ is the current parent of ‘C’.
    - It is not, so it’s a back-edge.



Step 5: Visit ‘E’ and Detect Back-Edges

Now at ‘C’, the algorithm explores it neighbors [A, D, E]:

- ‘D’ is visited. ‘D’ is the current parent of ‘C’?
  - It is, no action is taken.
- ‘E’ is not visited. The algorithm calls dfs_recursive(‘E’, ‘C’)
- ‘E’ is marked as visited, and ('C', 'E') is added to the result list.



Step 5: Visit ‘E’ and Detect Back-Edges

From 'E', the algorithm explores its neighbors [C, D]:

- 'C' is visited and is the current parent of ‘E’.
  - No action.
- 'D' is visited and is not the current parent.
  - This is another back-edge.